TriGuard: Enabling Oblivious Triangle Counting for Streaming Graphs

Songlei Wang
Shenzhen University
songlei.wang@szu.edu.cn

Yifeng Zheng
The Hong Kong Polytechnic University
yifeng.zheng@polyu.edu.hk

Yi Zhang
Sichuan University
yzhang@scu.edu.cn

Yuchuan Luo
National University of Defense Technology
luoyuchuan09@nudt.edu.cn

Cong Wang
City University Hong Kong
congwang@cityu.edu.hk

Abstract

Graph analysis enables the extraction of valuable insights from information-rich graphs that effectively capture the interactions among different entities in various scenarios. As a fundamental graph analysis task, triangle counting plays a crucial role in various graph analytic tasks such as community detection, fraud detection, and structural pattern analysis. Driven by the growing privacy concerns in graph-based applications, works on privacy-aware triangle counting over graphs have emerged in recent years. However, most existing approaches are tailored for static graphs and cannot handle streaming graphs that are dynamic and unbounded in nature. Moreover, prior approaches fail to distinguish between the two types of directed triangles: cycle triangles and flow triangles, both of which are essential for understanding directional relationships and structural characteristics of graphs. In this paper, we present TriGuard, the first system framework enabling oblivious and accurate counting of directed triangles on streaming graphs. Through customized designs, TriGuard allows cloud servers to obliviously and continuously count both cycle and flow triangles on an outsourced streaming graph, while hiding the graph topology. Experimental results on real-world graph datasets show that compared to a baseline leveraging generic secure multiparty computation, TriGuard achieves an improvement of up to 285× in runtime and cuts communication cost by up to 95%. Furthermore, in contrast to prior work that trades accuracy for privacy, TriGuard ensures accuracy while incurring only about 20% additional runtime overhead.

1 Introduction

Streaming graphs are dynamic graphs in which vertices and edges are continuously generated. They have gained significant adoption in numerous domains (such as social media, computer networks, and finance [26]), due to the ability to capture complex temporal interactions among entities. With the well-recognized benefits of cloud computing [21], outsourcing an unbounded streaming graph to the cloud for storage and query processing is a natural choice and has become increasingly common (e.g., [3, 13, 33]). However, such cloud-based service also poses threats to the privacy of the outsourced information-rich streaming graph which may contain sensitive information (e.g., relationships among different users in social networks, transactions among different customers in financial networks, and more). It is thus essential to provide privacy assurance in cloud-based streaming graph analysis services.

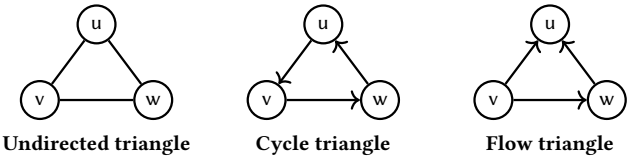


Figure 1: Illustration of different types of triangles in graphs.

Among others, *triangle counting*, which aims to compute the number of vertex triples that form triangles in a graph, is a fundamental graph analysis task that supports a wide range of downstream tasks, including community detection, node importance assessment, and structural modeling of complex networks [14, 41, 43]. It also underpins practical applications such as spam detection [5] and fraud detection [45]. For example, in social networks where user interactions evolve continuously across participants, triangle counting provides a core metric for evaluating community cohesion, influence propagation, and trust formation [26]. In financial systems, transaction data arrive as an unbounded stream graph, and triangle counting captures structural indicators of connectivity and tightly connected transaction patterns, reflecting the stability and correlation patterns within financial ecosystems [35].

Enabling triangle counting with graph privacy protection has received increasing attention in recent years. Existing studies [16, 18–20, 30, 39, 45] focus on the relatively uncommon setting of *static decentralized graphs*. In this setting, a fixed global graph is distributed among a set of users, where each user corresponds to a vertex and locally maintains a fixed neighbor list. Most of these works [16, 18–20, 30, 45] rely on local differential privacy (DP) [9] to protect the underlying static graph topology, incurring substantial utility loss in exchange for only modest privacy guarantees. An exception is the work of Wang *et al.*[39], which achieves lossless triangle counting accuracy by leveraging a secret-sharing framework[1]. To the best of our knowledge, no prior work has explored secure triangle counting over outsourced *streaming* graphs. It is worth noting that methods designed for static graphs are unsuitable for streaming settings, because they typically assume that the graph structure remains *fixed* throughout the triangle counting process.

In this paper, we propose TriGuard, the *first* system framework for secure triangle counting over outsourced streaming graphs. Unlike prior approaches which only work with undirected graphs, we consider a more practical setting where the (streaming) graph is

directed [14, 26, 34], which means that each edge has a direction and is associated with a source vertex and a destination vertex. As shown in Fig. 1, directed edges can form two types of triangles: *cycle triangles* and *flow triangles* [29]. TriGuard supports secure counting of both these types of triangles. At a high level, TriGuard allows a graph owner to outsource a streaming graph in protected form to the cloud, which will continuously perform triangle counting and return the results. TriGuard ensures that the structure of the streaming graph remains hidden throughout the entire counting process. In building TriGuard, we leverage a distributed trust model, where three cloud servers from different trust domains jointly empower the secure cloud-based streaming graph analysis service. Such a model has seen increasing adoption in building secure systems for different applications [6, 8, 24, 37], as well as in industry [11, 31, 32]. Accordingly, we resort to lightweight secret sharing techniques to support tailored secure computation for continuous triangle counting among the servers. To realize *continuous* triangle counting, we follow the standard practice in the cleartext domain and adopt the sliding-window model [14, 15, 26], where triangle counting is performed over each snapshot graph as specified by a sliding time window. In particular, the edges/vertices within the (sliding) time window form a snapshot graph for one triangle counting process.

Note that as edges emerge over time, the snapshot graphs formed by different time windows are also dynamic by nature and have different structures. Moreover, as *new* edges continuously appear and expire within each window, private search methods designed for static graphs remain unsuitable, since they require access to the complete graph for index construction or structural preprocessing.

We start with considering how to properly represent the streaming graph. We observe that the *edge list-based representation* [14, 26, 34] is a well-suited choice, where each edge is stored as an independent tuple and vertices are implicitly represented by their IDs within these tuples. With this representation, newly arriving edges can simply be appended to the secret-shared edge list maintained on the server side.

With this as a basis, we then consider how to support secure triangle counting on the server side. We observe that the core of triangle counting lies in *determining the cardinalities of intersections or unions of the neighbor sets of an edge's source and destination vertices*. For example, the number of cycle triangles formed by an edge from vertex v to vertex w equals the cardinality of the intersection between v's in-neighbors and w's out-neighbors. Thus, the first step in secure triangle counting is to securely aggregate each vertex's in- and out-neighboring vertices. However, this aggregation is challenging under the edge-list structure in the sliding-window model, because vertices are represented only implicitly through their incident edges, and each edge is stored independently.

To address this challenge, our main idea is to formulate neighboring vertex aggregation as a message passing process between vertices and their neighbors, and so secure neighboring vertex aggregation reduces oblivious message passing. Here, oblivious message passing means that messages are propagated along edges while both the underlying graph structure and the message contents are hidden. Under this framework, the servers obliviously propagate each vertex's ID along its incident edges to neighboring vertices and then aggregate all received secret-shared IDs to obtain each vertex's secret-shared neighbor set. We observe that in the literature there exist oblivious message passing techniques [2, 24] for graphs, which also operate within the secret-sharing domain.

However, directly applying the state-of-the-art oblivious message passing technique [24] does not yield a working solution in our setting for several reasons. First, it requires prior knowledge of the total number of vertices, which is incompatible with streaming graphs whose vertex set may grow continuously over time. Second, it only supports message passing in the *forward direction* of edges, and thus can only enable secure computation over in-neighbors. In our setting, however, message passing in the *backward direction* is also necessary to support secure aggregation over out-neighbors. Third, it is designed for iterative multi-round message passing, whereas neighboring vertex aggregation in our protocol only requires a single round. To overcome these limitations, we develop a tailored oblivious message passing method for secure neighboring vertex aggregation. Our method accommodates dynamically growing vertex sets, supports both forward and backward message propagation, and is specialized for one-round aggregation.

Another challenge lies in how to encode vertex IDs during oblivious message passing. Under (tailored) oblivious message passing, messages received by each vertex must be aggregated (i.e., summed) to hide message propagation patterns and the underlying graph structure. However, in our setting, directly summing scalar vertex IDs would obscure individual neighboring vertices, which are required for subsequent triangle counting. Therefore, message aggregation must preserve neighboring vertex information while still hiding message propagation patterns and graph structure. To achieve this, we design a shared one-hot vector encoding scheme for vertex IDs. Under this encoding, during the oblivious message passing process, neighboring vertex IDs can be directly summed into in- and out-neighbor vectors that fully capture all neighbors while hiding both message passing patterns and the underlying graph structure. As a result, computing the cardinalities of intersections and unions of neighbor sets reduces to element-wise products and dot-product operations over these vectors, thereby enabling secure triangle counting.

Contributions. Our main contributions are as follows:

- We present the *first* system framework for oblivious and accurate triangle counting on streaming directed graphs. To the best of our knowledge, TriGuard is the first framework capable of handling streaming directed graphs while securely supporting counting of both cycle and flow triangles.
- We propose a secure neighbor aggregation method for securely aggregating each vertex's in- and out-neighboring vertices during the secure triangle counting process. The method relies on a new single-round oblivious message passing protocol with bidirectional edge propagation, which may be of independent interest.
- We provide formal security analysis and conduct extensive experiments on four real-world graph datasets. Compared to a baseline built on the popular generic secure multi-party computation (MPC) framework MP-SPDZ [23], TriGuard achieves up to a 285× speedup in runtime and a 95% reduction in server-side communication cost. Furthermore, in contrast to the closest state-of-the-art method [45], which leverages DP and trades accuracy for privacy, TriGuard is accuracy-preserving, with only about 20% additional runtime overhead.

2 Related Work

In the literature, a line of work has been devoted to privacy-aware triangle counting. However, existing studies [16, 18–20, 30, 39, 45] primarily target *static decentralized graphs*. To count triangles, they typically require users to either interact with the servers in multiple rounds using information derived from their static local views [16, 18–20, 30, 45], or directly upload their encrypted local views to the servers for secure computation [39]. Moreover, most prior works [16, 18–20] adopt DP [9] to protect the underlying graph topology. However, DP-based mechanisms inevitably introduce non-trivial utility loss, even under moderate privacy budgets. For example, with a privacy budget of $\varepsilon = 1$, the latest DP-based method [16] can incur up to a 10% mean relative error in triangle counts. Recent works [30, 45] combine MPC with DP to improve accuracy while preserving privacy; nevertheless, under the same privacy setting ($\varepsilon = 1$), their mean relative error still remains around 5%. Therefore, accuracy degradation remains a major obstacle to the practical deployment of these methods.

In addition, most existing methods consider only *undirected* graphs. In contrast, real-world graphs can actually be directed and contain two distinct types of triangles—*cycle triangles* and *flow triangles* [29]. Because edge direction is crucial for characterizing structural properties in directed graphs [26], triangle counts produced by undirected methods fail to capture directional topology. Wang *et al.* [39] rely solely on MPC to support edge direction. However, their method is limited to static graphs, does not distinguish between cycle triangles and flow triangles, and does not consider dynamic graphs. To the best of our knowledge, no prior work has addressed privacy-aware triangle counting for streaming graphs. Our design presents the first secure and accurate triangle counting method tailored for streaming graphs.

Beyond triangle counting, most existing works on other secure graph analytic tasks (such as privacy-preserving shortest-path queries [10, 12] and privacy-preserving subgraph matching [17, 44]) also only consider *static* graphs. Only a limited number of works consider *streaming* graphs. In particular, Wang *et al.* [38] study privacy-preserving time-constrained pattern detection over streaming graphs, while Wu *et al.* [42] investigate privacy-preserving node- and edge-based queries on dynamic attributed graphs. These methods target fundamentally different graph analytic tasks and cannot be applied to secure triangle counting on streaming graphs.

3 Preliminaries

3.1 Triangle Counting on Streaming Graphs

Following the latest advances of streaming graph analysis in the plaintext domain [26], we consider a directed and unweighted streaming graph.

Definition 1. ***Streaming graph.*** *A streaming directed graph is a continually growing sequence of edges, denoted as $G = \{e_{i,j}, \dots\}$, where $e_{i,j} = (v_i, v_j, t)$ represents a directed edge from vertex v_i to vertex v_j that arrives at time point t.*

Following [26], we do not consider multiple edges with the same source and destination. For a directed edge $e_{i,j} = (v_i, v_j)$, vertex v_i is defined as the source and v_j as the destination. We refer to v_i as an in-neighbor of v_j, and to v_j as an out-neighbor of v_i. We denote the in-neighbors of vertex v by $\mathbf{N}_v^-$, and the out-neighbors by $\mathbf{N}_v^+$. In addition, throughout this paper we use $[N]$ to denote the set $\{1, \dots, N\}$. Building on prior work in the plaintext domain [14, 26] and motivated by the increasing importance of recent data, we focus on triangle counting over the most recent edges, represented by a *snapshot graph*. This model avoids unnecessary computation and has been widely adopted in plaintext-domain streaming graph analysis [14, 15, 25, 26].

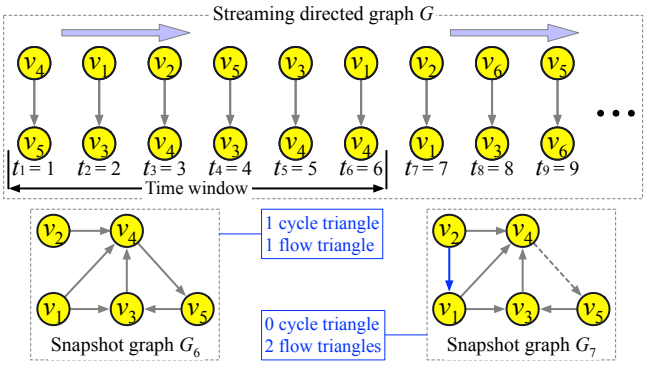


Figure 2: Illustration of triangle counting on streaming graph G with a time window of size 6.

Definition 2. ***Snapshot graph.*** *Given a streaming graph G and a time window size W, the snapshot graph G_t at the current time point t is the subgraph formed by all edges whose timestamps fall within the interval $(t - W, t]$.*

In this paper, we focus on continuous triangle counting over streaming graphs, which maintains the number of triangles in the current snapshot [14].

Definition 3. ***Triangle counting on a streaming directed graph.*** *Given a streaming directed graph G and a time window of size W, triangle counting reports, at each time point t, the number of cycle triangles and flow triangles in the current snapshot graph G_t.*

Example 1. *For clarity, we illustrate triangle counting on a streaming graph G with a time window of size 6 in Fig. 2. At time point $t = 6$, the snapshot graph G_6 consists of the edges $\{e_{4,5}, e_{1,3}, e_{2,4}, e_{5,3}, e_{3,4}, e_{1,4}\}$, which contains one cycle triangle $\{v_3, v_4, v_5\}$ and one flow triangle $\{v_1, v_3, v_4\}$. At time point $t = 7$, the snapshot graph G_7 consists of the edges $\{e_{1,3}, e_{2,4}, e_{5,3}, e_{3,4}, e_{1,4}, e_{2,1}\}$, which contains no cycle triangles and two flow triangles $\{v_1, v_2, v_4\}$ and $\{v_1, v_3, v_4\}$.*

3.2 Replicated Secret Sharing

Given a private value $x \in \mathbb{Z}_{2^k}$, 2-out-of-3 replicated secret sharing (RSS) [1] splits it into three shares $\langle x \rangle_1, \langle x \rangle_2, \langle x \rangle_3 \in \mathbb{Z}_{2^k}$, where $\langle x \rangle_1$ and $\langle x \rangle_2$ are random and $\langle x \rangle_3 = x - \langle x \rangle_1 - \langle x \rangle_2$. The pairs $(\langle x \rangle_1, \langle x \rangle_2)$, $(\langle x \rangle_2, \langle x \rangle_3)$, and $(\langle x \rangle_3, \langle x \rangle_1)$ are distributed to three parties P_1, P_2, P_3, respectively. For convenience, when indexing parties (or shares), $i - 1$ and $i + 1$ denote the previous and next party (or share) with wrap-around. For example, $P_{3+1} = P_1$ and $P_{1-1} = P_3$. Accordingly, the shares held by P_i can be written as $(\langle x \rangle_i, \langle x \rangle_{i+1})$, and the RSS of x is denoted $[\![x]\!]$.

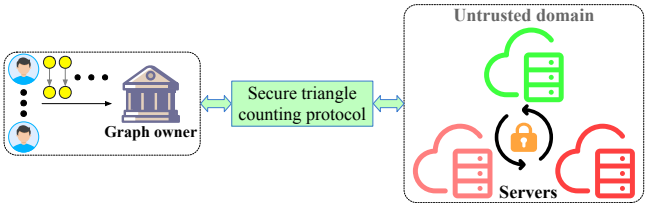


Figure 3: The system architecture of TriGuard.

The basic operations in the RSS domain are: (1) *Secret-shared addition/subtraction.* Operations on secret-shared values require only local computation. To compute $[\![u]\!] = [\![x \pm y]\!]$, each P_i computes $\langle u\rangle_i = \langle x\rangle_i \pm \langle y\rangle_i$, $\langle u\rangle_{i+1} = \langle x\rangle_{i+1} \pm \langle y\rangle_{i+1}$. (2) *Scalar multiplication.* Multiplying a public scalar η with a secret-shared value also requires only local computation: $\langle u\rangle_i = \eta \cdot \langle x\rangle_i$, $\langle u\rangle_{i+1} = \eta \cdot \langle x\rangle_{i+1}$. (3) *Secret-shared multiplication.* Multiplying two secret-shared values requires one round of online communication. To compute $[\![u]\!] = [\![x \cdot y]\!]$, each P_i first computes locally: $\langle u\rangle_i = \langle x\rangle_i \cdot \langle y\rangle_i + \langle x\rangle_i \cdot \langle y\rangle_{i+1} + \langle x\rangle_{i+1} \cdot \langle y\rangle_i$. This produces a 3-out-of-3 additive secret sharing of u, where each P_i holds only $\langle u\rangle_i$. To continue computations in the RSS domain, a re-sharing step is required [1].

4 System Overview

4.1 System Architecture

Fig. 3 shows the architecture of TriGuard. In the system, the graph owner collects edges of a (streaming) graph from its users and aims to continuously monitor the counts of cycle and flow triangles over the dynamically evolving graph. This design is motivated by the benefits of computation outsourcing, which allow the graph owner to offload the management of the continuous streaming graph and triangle counting to external servers, thereby reducing local infrastructure costs [22, 30, 38]. A representative example is network supervision, where a cybersecurity center leverages cloud resources to continuously process dynamic IP-access data from multiple hosts and perform triangle counting. In such network graphs, hosts can be regarded as vertices and IP-access events as edges. A triangle corresponds to a loop [27]; therefore, a sudden increase in triangle counts at a given time may indicate abnormal access patterns, such as routing anomalies or lateral movement. In practice, such an outsourcing paradigm has been widely adopted in dynamic graph search systems (e.g., [3, 13, 33]). However, due to the sensitive nature of the data contained in the streaming graph, it is critical to integrate privacy protection into outsourced graph services to safeguard both the graph and the resulting counts.

TriGuard leverages a distributed trust model, where the computational power of the party providing the counting service is divided across three servers (S_1, S_2, and S_3) from different trust domains. Such a distributed trust model has gained increasing adoption in recent security designs (e.g., [6, 8, 24, 36, 37]) and is also widely implemented in industry [11, 31, 32].

4.2 Threat Model and Security Guarantees

Threat model. Similar to prior studies that employ the multi-server setting for security designs [6, 24, 36], we adopt a semi-honest, non-colluding adversary model. Under this model, each server may *individually* attempt to infer private information beyond the leakage specified in the security guarantees during the triangle-counting services. In practice, the non-colluding servers may be instantiated as commercial cloud providers operated by competing companies (e.g., Amazon, Microsoft, and Google). The competitive landscape among these providers creates strong incentives to act independently and avoid collusion. Moreover, since our primary goal is to protect the privacy of the streaming graph from the servers, we assume that the graph owner is a trustworthy party.

Security guarantees. Under the above threat model, TriGuard guarantees that each server learns only the timestamp of each edge and limited metadata, and nothing beyond this information. Timestamps are treated as public, since servers can already infer them from the upload time of edges. This assumption aligns with prior security designs for time-series data (e.g., [8]). The metadata revealed consists solely of the number of edges in the current streaming graph and the length of the query-specified time window.

5 Technical Overview

In a streaming graph, edges arrive continuously and independently, while vertices are implicitly defined by their incident edges. Accordingly, we adopt the widely used edge-list representation [14, 26] to model the streaming graph, as formalized in Definition 1. Cycle and flow triangle counting is then performed on each snapshot of the streaming graph induced by a sliding time window. We observe that the core of cycle and flow triangle counting lies in *computing the cardinalities of intersections or unions of the neighbor sets of an edge's source and destination vertices.* For example, the number of cycle triangles induced by an edge from vertex v to vertex w equals the cardinality of the intersection between v's in-neighbor set and w's out-neighbor set. Consequently, the first step in triangle counting is to aggregate the in- and out-neighbor sets of each vertex. Triangle counting then reduces to computing intersections or unions over these aggregated neighbor sets.

To achieve efficient and secure triangle counting over streaming graphs, our system leverages the lightweight cryptographic primitive RSS. We first consider how to obliviously aggregate the in- and out-neighbor sets of each vertex in the secret-sharing domain. This aggregation is non-trivial under the edge-list representation in the sliding-window model, since vertices are represented only implicitly via their incident edges, and each edge is stored independently. To address this challenge, we formulate neighboring vertex aggregation as a *message passing* process between vertices and their neighbors. Under this framework, each vertex propagates its ID along its incident edges to neighboring vertices, which then aggregate all received IDs to construct their neighbor sets. To secure this process, TriGuard requires that message propagation along edges be performed while hiding both the underlying graph structure and the message contents throughout the process.

We note that there exist *oblivious message passing* techniques [2, 24] operating in the secret sharing domain for secure graph analytics. However, directly adopting the state-of-the-art oblivious message passing technique [24] does not provide a working and satisfactory solution for our considered streaming graph setting. First, it requires prior knowledge of the total number of vertices, which is infeasible for streaming graphs due to their unbounded

nature. Second, it supports message passing only in the *forward direction* of edges, enabling secure computation of in-neighbors but not out-neighbors, as message propagation in the *backward direction* is not supported. Third, it is designed to support multiple rounds of oblivious message passing, whereas neighboring vertex aggregation in our setting requires only a single round. To address these limitations, we develop a tailored oblivious message passing method to support secure neighboring vertex aggregation in the context of streaming graphs. Our method supports oblivious message propagation in both the *forward and backward* directions of edges and is simplified to a single round of message passing.

It is worth noting that, under (tailored) oblivious message passing, messages received by each vertex must be aggregated (i.e., summed) in order to hide message propagation patterns and the underlying graph structure. However, in our setting, directly summing scalar messages would prevent the extraction of individual neighboring vertex IDs from the aggregated result, which is required for subsequent triangle counting. Therefore, we must ensure that message aggregation does not destroy neighboring vertex information while still hiding message propagation patterns and graph structure. To address this challenge, we design a shared one-hot vector encoding mechanism for vertex IDs. Under this encoding, during the oblivious message passing process, neighboring vertex identifiers can be directly summed into in- and out-neighbor vectors that fully capture all neighbors while hiding both message passing patterns and the underlying graph structure. The cardinalities of intersections and unions of neighbor sets then reduce to element-wise products and dot-product operations over these vectors, thereby enabling secure triangle counting.

In what follows, we first introduce the preprocessing and protection of the streaming graph (Section 6), and then present a secure neighboring-vertex aggregation method (Section 7), in which the servers obliviously aggregate each vertex's in- and out-neighbors to obtain the corresponding neighbor vectors. We then introduce an oblivious triangle-counting method based on these secret-shared neighbor vectors (Section 8).

6 Preprocessing and Protection of the Streaming Graph

As introduced above, we leverage the concept of oblivious message passing to perform secure triangle counting on each snapshot of the streaming graph. We note that during oblivious message passing, *secure sorting* is a frequently invoked subroutine, and the state-of-the-art secure sorting protocol [4] requires sort keys to be represented in *bit-decomposed form*. Specifically, each sort key x should be represented as a bit-decomposed vector $[\vec{x}[1], \ldots, \vec{x}[k]]$, where $x = \sum_{i=1}^{k} 2^{i-1} \cdot \vec{x}[i]$. Moreover, to accommodate each vertex's neighboring vertex IDs without leaking vertex degrees during neighbor aggregation, and to support efficient intersection and union operations in the secret-sharing domain, we further encode each private value as a one-hot vector. Concretely, for a locally generated edge $e_{i,j} = (v_i, v_j, t)$, the graph owner encodes vertices v_i and v_j into both their bit-decomposed representations (denoted as $\vec{v}_i$ and $\vec{v}_j$) and their one-hot vectors (denoted as $\mathbf{v}_i$ and $\mathbf{v}_j$). Each element of these representations is then secret-shared and distributed to the servers, together with the public timestamp t.

After obtaining the secret-shared streaming graph, the servers first derive the snapshot graph at each time point. Specifically, since the timestamp of each edge in the secret-shared streaming graph is available to the servers, they can directly construct the current snapshot graph $[\![G_t]\!] = \{[\![e_{i,j}]\!], \ldots\}$ by checking whether each edge's timestamp falls within the interval $(t - W, t]$, where t denotes the current time point and W denotes the window size.

In addition, the oblivious message passing technique requires the input graph to be represented as a data-augmented graph (DAG) list. In this representation, each entity (a vertex or an edge) is encoded as a tuple with three components: "src" (source vertex ID), "dst" (destination vertex ID), and "isV" (a flag indicating whether the tuple corresponds to a vertex or an edge). For example, a vertex v_i is encoded as $(v_i, v_i, 1)$, while a directed edge $e_{i,j}$ is encoded as $(v_i, v_j, 0)$. Therefore, the servers first locally convert each secret-shared snapshot graph from the edge-list representation into the DAG-list representation as follows. For each $[\![e_{i,j}]\!] \in [\![G_t]\!]$, the servers first extract its source and destination vertex $[\![v_i]\!]$ and $[\![v_j]\!]$, and then convert the edge into three tuples in the DAG-list: $([\![v_i]\!], [\![v_i]\!], [\![1]\!])$, $([\![v_j]\!], [\![v_j]\!], [\![1]\!])$, and $([\![v_i]\!], [\![v_j]\!], [\![0]\!])$. Here, the secret shares of $[\![1]\!]$ and $[\![0]\!]$ can be generated locally in an offline manner.

In the subsequent description, we use "**src**" and "**dst**" to denote the one-hot encoded vertex IDs, and "$\overrightarrow{\text{src}}$" and "$\overrightarrow{\text{dst}}$" to denote their bit-decomposed forms. Note that since each vertex may be adjacent to multiple edges, this conversion to a DAG-list will produce duplicate vertex entries. During the subsequent secure neighbor aggregation phase (see Section 7.2 and Fig. 6), any duplicate vertices in the DAG-list are automatically ignored by the servers. For simplicity, we continue to use $[\![G_t]\!]$ to denote the secret-shared DAG-list, and let N denote the total number of tuples in the list.

7 Secure Aggregation of Neighboring Vertices

7.1 Design Rationale

Our secure neighbor aggregation method takes as input the secret-shared DAG-list and outputs an updated DAG-list in which each vertex tuple is augmented with its secret-shared out-neighbors and in-neighbors. The core idea can be summarized under the Scatter-Gather framework as follows: (1) for out-neighbor aggregation, the servers obliviously propagate each edge tuple's destination vertex ID (encoded as a one-hot vector) to the tuple corresponding to its source vertex (Scatter), and then obliviously aggregate the received IDs to obtain the out-neighbor vector (Gather); and (2) for in-neighbor aggregation, the servers obliviously propagate each edge tuple's source vertex ID to the tuple corresponding to its destination vertex (Scatter), and then obliviously aggregate the received IDs to obtain the in-neighbor vector (Gather). In these neighbor vectors, the positions of elements with value 1 correspond to the IDs of the neighboring vertices.

Example 2. *Fig. 4 illustrates the underlying framework of our secure neighbor aggregation method. The aggregation produces the in-neighbor vectors* $v_1 : [0, 0, 0, 1]$, $v_2 : [0, 0, 0, 0]$, $v_3 : [1, 0, 0, 0]$, *and* $v_4 : [0, 1, 1, 0]$, *as well as the out-neighbor vectors* $v_1 : [0, 0, 1, 0]$, $v_2 : [0, 0, 0, 1]$, $v_3 : [0, 0, 0, 1]$, *and* $v_4 : [1, 0, 0, 0]$. *For each edge, the number of triangles it participates in can then be computed via element-wise vector operations between the corresponding neighbor*

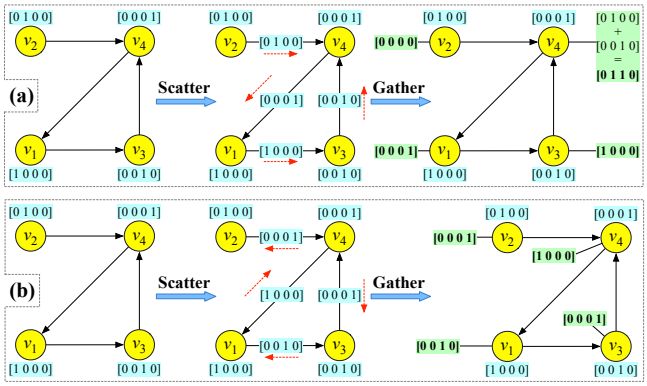


Figure 4: Example of the underlying framework for our secure neighbor aggregation method: (a) in-neighbor aggregation; (b) out-neighbor aggregation.

vectors. For example, the number of cycle triangles induced by edge $e_{1,3}$ *is the dot product between the in-neighbor vector of its source vertex* v_1, $[0, 0, 0, 1]$, *and the out-neighbor vector of its destination vertex* v_3, $[0, 0, 0, 1] : [0, 0, 0, 1] \cdot [0, 0, 0, 1] = 0 \cdot 0 + 0 \cdot 0 + 0 \cdot 0 + 1 \cdot 1 = 1$.

We observe that the state-of-the-art oblivious message passing technique [24] is a good starting point, as it operates in the secret-sharing domain for secure graph analytics. However, as discussed in Section 5, directly adopting this technique does not yield a satisfactory solution in the streaming graph setting. First, it requires prior knowledge of the total number of vertices, which is infeasible for streaming graphs due to their unbounded and dynamic nature. Second, it supports message passing only in the *forward direction* of edges, enabling secure aggregation of in-neighbors, but does not support message propagation in the *backward direction*, and therefore cannot support secure aggregation of out-neighbors.

To overcome these limitations, we develop our secure neighbor aggregation method based on a new oblivious message passing approach that is specifically tailored for triangle counting on streaming graphs. Fig. 5 illustrates the overall framework. To support message propagation in the backward direction, we observe that the DAG-list can be sorted in the reverse order of that used in secure forward message passing. Specifically, for out-neighbor aggregation, the servers start from the initial DAG-list and treat the destination vertex of each edge tuple as the message to be propagated. They then obliviously sort the DAG-list into the *source order*, where all outgoing edges of each vertex appear immediately before the vertex itself, and all vertices are arranged sequentially[1] (e.g., $e_{1,2}, v_1, e_{2,3}, e_{2,4}, v_2, \dots$). This sorting is achieved in two steps: (1) Obliviously sort the list by the "isV" field in ascending order, ensuring that all vertex tuples (isV = 1) appear after edge tuples (isV = 0). (2) Perform a stable oblivious sort by the "src" field in ascending order so that each vertex appears just after its outgoing edges. Under this order, the servers aggregate the messages from each tuple's preceding tuples to obtain each vertex's out-neighbors, i.e., by propagating vertex IDs in the *backward direction* of edges. However, this may result in redundant out-neighbors for some vertices. To obliviously remove these redundant entries, the servers reorder the DAG-list into the *vertex order*, where all vertex tuples appear consecutively before all edge tuples (e.g., $v_1, v_2, \dots, e_{1,2}, e_{2,3}, \dots$). This is achieved by first sorting by src in ascending order, followed by sorting by isV in descending order. Under this order, the servers can obliviously remove redundant out-neighbors for each vertex by subtracting the out-neighbor set of the previous vertex tuple.

[1]The definition of source order here differs from that in [24], which defines the source order such that all outgoing edges of each vertex appear immediately *after* the vertex itself, and all vertices are arranged sequentially.

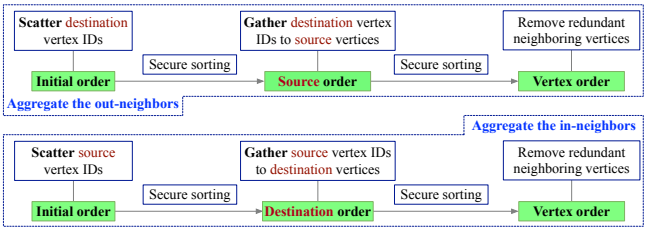


Figure 5: Framework of secure neighbor aggregation.

The process for aggregating in-neighbors is similar. Here, each edge tuple's source vertex is stored as the message to be passed. The servers then obliviously sort the DAG-list into the *destination order*, where all incoming edges of a vertex appear immediately before the vertex itself, and all vertices are arranged sequentially (e.g., $e_{2,1}, v_1, e_{1,2}, e_{3,2}, v_2, \dots$). This is accomplished by sorting on the "isV" field (ascending) and then on the "dst" field (ascending). Under this order, the servers aggregate the messages of each tuple's preceding tuples to obtain each vertex's in-neighbors, i.e., by propagating vertex IDs in the *forward direction* of edges. The DAG-list is then converted back to the vertex order to remove redundant in-neighbors for each vertex.

7.2 Detailed Protocol

We now introduce the details of our secure neighboring vertices protocol, which consists of three phases: initialization, secure out-neighbor aggregation, and secure in-neighbor aggregation.

Initialization. We now describe how the servers can obliviously and efficiently aggregate each vertex's neighboring vertices using the framework shown in Fig. 5. At the beginning, the servers augment each tuple in $[\![G_t]\!]$ with four secret-shared vectors:

- $[\![\mathbf{N}^+_{\mathrm{src}}]\!]$: stores the source vertex's out-neighbors.
- $[\![\mathbf{N}^-_{\mathrm{src}}]\!]$: stores the source vertex's in-neighbors.
- $[\![\mathbf{N}^+_{\mathrm{dst}}]\!]$: stores the destination vertex's out-neighbors.
- $[\![\mathbf{N}^-_{\mathrm{dst}}]\!]$: stores the destination vertex's in-neighbors.

Each of these vectors is initialized as a secret-shared zero *vector*, whose length matches that of the one-hot vectors corresponding to the "$[\![\mathbf{src}]\!]$" and "$[\![\mathbf{dst}]\!]$" fields in $[\![G_t]\!]$. After neighbor aggregation, the elements in the vectors corresponding to actual neighbors are obliviously set to 1, while all other elements remain 0.

The secure out-neighbor and in-neighbor aggregations are presented in Algorithm 1 (denoted as secNout) and Algorithm 2 (denoted as secNin), respectively. Both algorithms follow a common Set − Gather structure: for out-neighbor aggregation, the servers set each edge's "$[\![\mathbf{dst}]\!]$" field as the message to be passed, and then aggregate all messages received by each vertex from its outgoing edges as its out-neighbors, storing them in $\mathbf{N}^+_{\mathrm{src}}$; for in-neighbor

Algorithm 1 Secure Out-neighbor Aggregation

Input: Secret-shared DAG-list $[\![G_t]\!]$.
Output: Secret-shared out-neighbors of each vertex.

1: $[\![G_t[i].\mathbf{N}^+_{\text{src}}]\!]$=$[\![G_t[i].\mathbf{dst}]\!] \cdot [\![\neg G_t[i].\text{isV}]\!], i \in [N]$. % Scatter
2: $[\![G^S_t]\!]$=secSort(secSort($[\![G_t]\!]$, $[\![\text{isV}]\!]$, ↑), $[\![\overrightarrow{\text{src}}]\!]$, ↑).%Source order
3: $[\![G^S_t[i].\mathbf{N}^+_{\text{src}}]\!] = \sum_{j\in[1,i]} [\![G^S_t[j].\mathbf{N}^+_{\text{src}}]\!], i \in [N]$. % Gather
4: $[\![G^V_t]\!]$=secSort(secSort($[\![G^S_t]\!]$, $[\![\overrightarrow{\text{src}}]\!]$, ↑), $[\![\text{isV}]\!]$, ↓).%Vertex order
5: $[\![G^V_t[i].\mathbf{N}^+_{\text{src}}]\!] - = [\![G^V_t[i-1].\mathbf{N}^+_{\text{src}}]\!], i \in [N-E, 2]$.
6: **return** Vertices' out-neighbors $[\![G^V_t[i].\mathbf{N}^+_{\text{src}}]\!]$, $i \in [N-E]$.

aggregation, the servers set each edge's "**src**" field as the message to be passed, and then aggregate all messages received by each vertex from its incoming edges as its in-neighbors, storing them in $\mathbf{N}^-_{\text{dst}}$.

Secure out-neighbor aggregation (Algorithm 1). The servers first set each edge tuple's "**dst**" as the message to be passed and store it in "$\mathbf{N}^+_{\text{src}}$" (line 1). Here, the notation "¬" denotes the logical NOT operator, which can be locally computed: one pair of shares is set as $\langle \text{isV}\rangle_1 = 1 - \langle \text{isV}\rangle_1$, while the others are set as $\langle \text{isV}\rangle_2 = -\langle \text{isV}\rangle_2, \langle \text{isV}\rangle_3 = -\langle \text{isV}\rangle_3$. Since vertex tuples have isV = 1 and edge tuples have isV = 0, element-wise multiplication between ¬isV and **dst** ensures that only edge tuples store their **dst** in $\mathbf{N}^+_{\text{src}}$, while the $\mathbf{N}^+_{\text{src}}$ of vertex tuples remains a zero vector.

Next, the servers securely sort the DAG-list into the source order (line 2). Here, we employ a state-of-the-art secure stable sorting protocol, denoted as secSort [4]. The notation "↑" indicates that sorting is performed in ascending order with respect to the specified keys. Since secSort requires the sorting keys to be in bit-decomposed form, we use the bit-decomposition of src and dst as the sorting keys, i.e., $\overrightarrow{\text{src}}$ and $\overrightarrow{\text{dst}}$.

In the source order, all outgoing edges of each vertex appear immediately before the vertex itself, while all vertices are arranged sequentially. This enables the servers to aggregate $\mathbf{N}^+_{\text{src}}$ from each tuple's preceding tuples into their own $\mathbf{N}^+_{\text{src}}$ (line 3). However, this process also accumulates extra values propagated from earlier tuples in the DAG-list. To remove these redundant values, the servers further sort the DAG-list from source order into vertex order (line 4), and then eliminate the extra entries to obtain the correct out-neighbors for each vertex (line 5). Here, the notation "↓" indicates that the sorting is performed in descending order with respect to the specified keys. $[N-E, 2]$ denotes the set $\{N-E, N-E-1, \cdots, 2\}$, and E represents the number of edges in the snapshot graph, which is publicly known to the servers. N represents the number of tuples in the DAG-list. Since each edge in the initial snapshot graph is expanded into three tuples, we have $N = 3E$. Therefore, N does not reveal any additional information to the servers. Because the DAG-list in vertex order places all edge tuples after the vertex tuples, the last E tuples correspond to edges. Therefore, only the first $N-E$ tuples (i.e., the vertices) are considered when computing out-neighbors, while the $\mathbf{N}^+_{\text{src}}$ fields of the last E edge tuples are reset to zero vectors.

Example 3. *Fig. 6 illustrates the process of secure out-neighbor aggregation on the snapshot G_6 from Fig. 2. The input is a DAG-list in its initial order, where edge and vertex tuples appear in the order of their emergence. Each edge tuple's $\mathbf{N}^+_{\text{src}}$ is initialized to its corresponding* **dst** *value and treated as the message to be propagated to the tuple corresponding to its source vertex, i.e., the out-neighbors. The DAG-list is then sorted into the source order, enabling the servers to aggregate the $\mathbf{N}^+_{\text{src}}$ from preceding tuples into each tuple's own $\mathbf{N}^+_{\text{src}}$. Subsequently, the DAG-list is sorted from the source order into the vertex order, allowing the system to eliminate redundant entries and obtain the correct out-neighbors for each vertex. The resulting output shows that each vertex tuple's $\mathbf{N}^+_{\text{src}}$ stores its out-neighbor vector, i.e., $\mathbf{N}^+_{v_1} = \mathbf{v}_3 + \mathbf{v}_4$, $\mathbf{N}^+_{v_2} = \mathbf{v}_4$, $\mathbf{N}^+_{v_3} = \mathbf{v}_4$, $\mathbf{N}^+_{v_4} = \mathbf{v}_5$, and $\mathbf{N}^+_{v_5} = \mathbf{v}_3$. Since each vertex ID is encoded in one-hot vector form, the positions of elements with value 1 in the out-neighbor vector correspond to the IDs of the out-neighbor vertices. From the example, we can observe that for duplicate vertices, only one vertex obtains the neighbors, while the other duplicate vertices are obliviously ignored by the servers.*

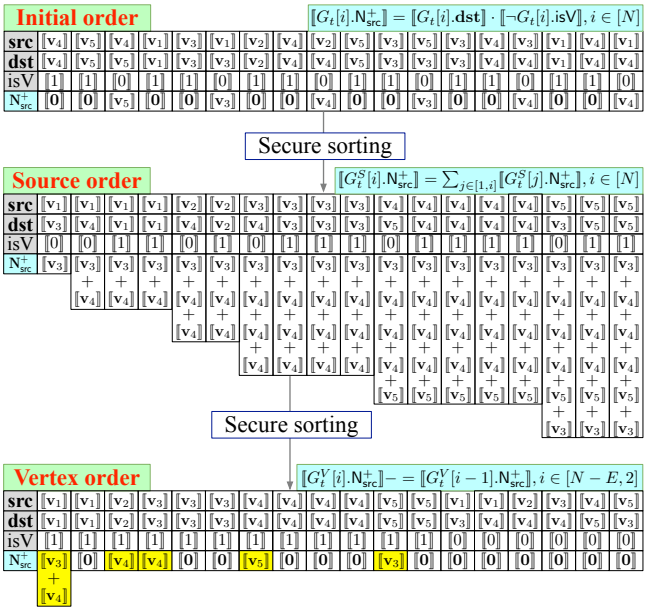


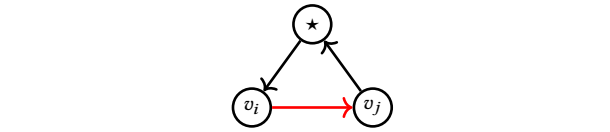
Figure 6: Illustration of secure out-neighbor aggregation on snapshot G_6 in Fig. 2.

Secure in-neighbor aggregation (Algorithm 2). The process is similar to that of secure out-neighbor aggregation in Algorithm 1. The servers first set each edge tuple's "**src**" as the message to be passed and store it in "$\mathbf{N}^-_{\text{dst}}$" (line 1). Next, the servers securely sort the DAG-list into the destination order (line 2). In this order, all ingoing edges of each vertex appear immediately before the vertex itself, while all vertices are arranged sequentially. This enables the servers to aggregate $\mathbf{N}^-_{\text{dst}}$ from each tuple's preceding tuples into their own $\mathbf{N}^-_{\text{dst}}$ (line 3). However, this process also accumulates extra values propagated from earlier tuples in the DAG-list. To remove these redundant values, the servers further sort the DAG-list from destination order into vertex order (line 4), and then eliminate the extra entries to obtain the correct in-neighbors for each vertex (line 5). For clarity, we provide an illustrative example of the secure in-neighbor aggregation in Appendix A of the full version [40].

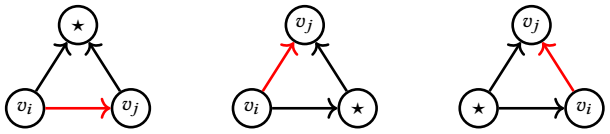
Algorithm 2 Secure In-neighbor Aggregation

Input: Secret-shared DAG-list $[\![G_t]\!]$.
Output: Secret-shared in-neighbors of each vertex.

1: $[\![G_t[i].\mathbf{N}^-_{\text{dst}}]\!]$=$[\![G_t[i].\text{src}]\!] \cdot [\![\neg G_t[i].\text{isV}]\!], i \in [N]$. % Scatter
2: $[\![G^D_t]\!]$ = secSort(secSort($[\![G_t]\!]$, $[\![\text{isV}]\!]$, ↑), $[\![\overrightarrow{\text{dst}}]\!]$, ↑). % Destination order
3: $[\![G^D_t[i].\mathbf{N}^-_{\text{dst}}]\!] = \sum_{j\in[1,i]} [\![G^D_t[j].\mathbf{N}^-_{\text{dst}}]\!], i \in [N]$. % Gather
4: $[\![G^V_t]\!]$=secSort(secSort($[\![G^D_t]\!]$, $[\![\overrightarrow{\text{src}}]\!]$, ↑), $[\![\text{isV}]\!]$, ↓). % Vertex order
5: $[\![G^V_t[i].\mathbf{N}^-_{\text{dst}}]\!]- = [\![G^V_t[i-1].\mathbf{N}^-_{\text{dst}}]\!], i \in [N-E]$.
6: **return** Vertices' in-neighbors $[\![G^V_t[i].\mathbf{N}^-_{\text{dst}}]\!], i \in [N-E]$.



(a) Cycle triangle induced by edge $e_{i,j}$



(b) Flow triangles induced by edge $e_{i,j}$

Figure 7: Triangles induced by edge $e_{i,j}$ under different structural roles.

8 Secure Triangle Counting via Neighbor Vectors

Overview. After the secure aggregation of neighboring vertices described in Section 7, each vertex tuple's $[\![\mathbf{N}^+_{\text{src}}]\!]$ stores its out-neighbors, and each vertex tuple's $[\![\mathbf{N}^-_{\text{dst}}]\!]$ stores its in-neighbors. Specifically, $\mathbf{N}^+_{\text{src}}$ and $\mathbf{N}^-_{\text{dst}}$ are vectors, where the positions of elements with value 1 correspond to the IDs of neighboring vertices. In this section, we describe how to compute secret-shared triangle counts based on the aggregated neighbor vectors.

Our idea is to first compute the number of triangles induced by each edge, and then aggregate these per-edge counts to obtain the overall triangle count in the snapshot graph. Therefore, we first identify the triangles induced by each edge $e_{i,j}$ under different structural roles, as illustrated in Fig. 7. From the figure, we can observe that the number of *cycle triangles* formed by edge $e_{i,j}$ equals the cardinality of the intersection between vertex v_i's in-neighbors and vertex v_j's out-neighbors. This can be formulated as the dot product (denoted by "$\cdot$") between vertex v_i's in-neighbor vector $\mathbf{N}^-_{v_i}$ and vertex v_j's out-neighbor vector $\mathbf{N}^+_{v_j}$:

$$\mathbf{N}^-_{v_i} \cdot \mathbf{N}^+_{v_j}. \tag{1}$$

However, the *flow triangles* induced by edge $e_{i,j}$ are more involved and can be categorized into three types. Accordingly, the number of flow triangles induced by $e_{i,j}$ equals the cardinality of the union of the following three intersections: (i) the intersection between the in-neighbors of v_i and the in-neighbors of v_j, (ii) the intersection between the out-neighbors of v_i and the in-neighbors of v_j, and (iii) the intersection between the out-neighbors of v_i and the out-neighbors of v_j. This can be formulated as

$$\text{sum}[(\mathbf{N}^-_{v_i} \odot \mathbf{N}^-_{v_j}) \vee (\mathbf{N}^+_{v_i} \odot \mathbf{N}^-_{v_j}) \vee (\mathbf{N}^+_{v_i} \odot \mathbf{N}^+_{v_j})]. \tag{2}$$

Here, "$\odot$" denotes element-wise multiplication between vectors, "$\vee$" denotes the element-wise OR operation, and sum$(\cdot)$ denotes the summation of all elements in a vector.

From Eq. 1 and Eq. 2, we observe that triangle counting can be reduced to element-wise vector operations. Addition and multiplication are naturally supported in the secret-sharing domain. However, the element-wise OR operation ($\vee$) is not natively supported under secret sharing. To facilitate the $\vee$ operation in the secret-sharing domain, we reformulate the element-wise OR between two vectors $\mathbf{a}$ and $\mathbf{b}$ as $\mathbf{a} \vee \mathbf{b} = \mathbf{a} + \mathbf{b} - \mathbf{a} \odot \mathbf{b}$.

Based on Eq. 1 and Eq. 2, it may appear straightforward to count the triangles induced by each edge using the neighbor vectors of its source and destination vertices, since the required operations are naturally supported in the secret-sharing domain. However, in the DAG-list produced by secure neighbor aggregation, the neighbor vectors are attached only to vertex tuples rather than to edge tuples. Therefore, we first propose a method that enables the servers to copy each vertex's neighbor vectors to the tuples corresponding to its incident edges, and then present our triangle counting method.

Obliviously copying neighbors to incident edges. Algorithm 3 presents the procedure, which takes as input the DAG-list in vertex order—i.e., the output of Algorithm 2—and outputs a new DAG-list in which each edge tuple is updated to store the out- and in-neighbor vectors of both its source and destination vertices. Note that in the output of Algorithm 2, each vertex's out-neighbors are stored only in $[\![\mathbf{N}^+_{\text{src}}]\!]$, while its in-neighbors are stored only in $[\![\mathbf{N}^-_{\text{dst}}]\!]$; the fields $[\![\mathbf{N}^+_{\text{dst}}]\!]$ and $[\![\mathbf{N}^-_{\text{src}}]\!]$ remain empty. To facilitate the copying of neighbor vectors to the incident edges' $\mathbf{N}^+_{\text{dst}}$ and $\mathbf{N}^-_{\text{src}}$, the servers first obliviously copy each vertex tuple's $\mathbf{N}^+_{\text{src}}$ to $\mathbf{N}^+_{\text{dst}}$, and $\mathbf{N}^-_{\text{dst}}$ to $\mathbf{N}^-_{\text{src}}$ (lines 1–2). The servers then sort the DAG-list into the source order (line 3), which enables them to obliviously copy each vertex's out- and in-neighbor vectors to the corresponding outgoing edges' $\mathbf{N}^+_{\text{src}}$ and $\mathbf{N}^-_{\text{src}}$ (line 4 and line 5, respectively). The oblivious copy operation is achieved by aggregating each tuple's $\mathbf{N}^+_{\text{src}}$ and $\mathbf{N}^-_{\text{src}}$ with those of its succeeding tuple's $\mathbf{N}^+_{\text{src}}$ and $\mathbf{N}^-_{\text{src}}$.

Algorithm 3 Obliviously Copying Neighbors to Incident Edges

Input: Secret-shared DAG-list $[\![G^V_t]\!]$ in vertex order.
Output: Secret-shared neighbor vectors of each edge.

1: $[\![G^S_t[i].\mathbf{N}^+_{\text{dst}}]\!] = [\![G^S_t[i].\mathbf{N}^+_{\text{src}}]\!], i \in [N-E]$. % Copy $\mathbf{N}^+_{\text{src}}$ to $\mathbf{N}^+_{\text{dst}}$
2: $[\![G^V_t[i].\mathbf{N}^-_{\text{src}}]\!] = [\![G^V_t[i].\mathbf{N}^-_{\text{dst}}]\!], i \in [N-E]$. % Copy $\mathbf{N}^-_{\text{dst}}$ to $\mathbf{N}^-_{\text{src}}$
3: $[\![G^S_t]\!]$ = secSort(secSort($[\![G^V_t]\!]$, $[\![\text{isV}]\!]$, ↑), $[\![\overrightarrow{\text{src}}]\!]$, ↑). %Source ord.
4: $[\![G^S_t[i].\mathbf{N}^+_{\text{src}}]\!]+ = [\![G^S_t[i+1].\mathbf{N}^+_{\text{src}}]\!] \cdot [\![\neg G^S_t[i].\text{isV}]\!], i \in [N-1,1]$.
5: $[\![G^S_t[i].\mathbf{N}^-_{\text{src}}]\!]+ = [\![G^S_t[i+1].\mathbf{N}^-_{\text{dst}}]\!] \cdot [\![\neg G^S_t[i].\text{isV}]\!], i \in [N-1,1]$.
6: $[\![G^D_t]\!]$ = secSort(secSort($[\![G^S_t]\!]$, $[\![\text{isV}]\!]$, ↑), $[\![\overrightarrow{\text{dst}}]\!]$, ↑).
7: $[\![G^D_t[i].\mathbf{N}^+_{\text{dst}}]\!]+ = [\![G^D_t[i+1].\mathbf{N}^+_{\text{src}}]\!] \cdot [\![\neg G^D_t[i].\text{isV}]\!], i \in [N-1,1]$.
8: $[\![G^D_t[i].\mathbf{N}^-_{\text{dst}}]\!]+ = [\![G^D_t[i+1].\mathbf{N}^-_{\text{dst}}]\!] \cdot [\![\neg G^D_t[i].\text{isV}]\!], i \in [N-1,1]$.
9: **return** Secret-shared neighbor vectors of each edge.

Algorithm 4 Secure Triangle Counting via Neighbor Vectors

Input: Secret-shared DAG-list $[\![G_t^D]\!]$ in destination order.
Output: Secret-shared counts of cycle and flow triangles.

1: Initialization: $[\![\triangle_c]\!] = [\![0]\!]$; $[\![\triangle_f]\!] = [\![0]\!]$.
2: $[\![G_t^V]\!]$=secSort(secSort($[\![G_t^D]\!]$, $[\![\overrightarrow{\text{src}}]\!]$, ↑), $[\![\text{isV}]\!]$, ↓). % Vertex order
3: **for all** $i \in [N-E, N]$ **do**
4: $\quad [\![\triangle_c]\!]$+ = $[\![G_t^V[i].\text{N}^-_{\text{src}}]\!] \cdot [\![G_t^V[i].\text{N}^+_{\text{dst}}]\!]$. % # of cycle triangles
5: $\quad [\![\mathbf{a}]\!] = [\![G_t^V[i].\text{N}^-_{\text{src}}]\!] \odot [\![G_t^V[i].\text{N}^-_{\text{dst}}]\!]$.
6: $\quad [\![\mathbf{b}]\!] = [\![G_t^V[i].\text{N}^+_{\text{src}}]\!] \odot [\![G_t^V[i].\text{N}^-_{\text{dst}}]\!]$.
7: $\quad [\![\mathbf{c}]\!] = [\![G_t^V[i].\text{N}^+_{\text{src}}]\!] \odot [\![G_t^V[i].\text{N}^+_{\text{dst}}]\!]$.
8: $\quad [\![\mathbf{d}]\!] = [\![\mathbf{a}]\!] + [\![\mathbf{b}]\!] - [\![\mathbf{a}]\!] \odot [\![\mathbf{b}]\!]$.
9: $\quad [\![\triangle_f]\!]$+ = sum($[\![\mathbf{d}]\!] + [\![\mathbf{c}]\!] - [\![\mathbf{d}]\!] \odot [\![\mathbf{c}]\!]$). % # of flow triangles
10: **end for**
11: **return** Secret-shared triangle counts $[\![\triangle_c]\!]$ and $[\![\triangle_f]\!]$.

Here, multiplication by $\neg G_t[i].\text{isV}$ ensures that only the edge tuples receive the copied neighbor vectors, while the vertex tuples remain unchanged. Next, the servers sort the DAG-list into the destination order (line 6), which enables them to copy each vertex's out- and in-neighbor vectors to the corresponding incoming edges' N^+_{dst} and N^-_{dst} fields (line 7 and line 8, respectively). For clarity, we provide an illustrative example of the secure in-neighbor aggregation process in Appendix B of the full version [40].

Computing triangle counts via neighbor vectors (Algorithm 4). After copying the neighbor vectors to the incident edges, each edge tuple's N^+_{src}, N^-_{src}, N^+_{dst}, N^-_{dst} respectively store the source vertex's out-neighbor vector, source vertex's in-neighbor vector, destination vertex's out-neighbor vector, and destination vertex's in-neighbor vector. The servers then compute the final secret-shared cycle triangle count $[\![\triangle_c]\!]$ and flow triangle count $[\![\triangle_f]\!]$. Our idea is to first compute the number of triangles induced by each edge and then aggregate these local counts to obtain the global triangle counts. However, since the output DAG-list of Algorithm 3 is in destination order, the servers cannot directly distinguish which tuples correspond to edges. Therefore, to identify the edge tuples, the servers obliviously sort the DAG-list into vertex order (line 2), in which the last E tuples (i.e., indices $i \in [N-E, N]$) correspond to edges. The servers then compute the number of cycle triangles induced by each edge according to Eq. 1 (line 4), and the number of flow triangles induced by each edge according to Eq. 2 (lines 5–9).

Finally, the servers return the secret-shared cycle and flow triangle counts to the graph owner for recovery. Note that each triangle consists of three edges; therefore, both cycle triangles and flow triangles are counted three times—once per incident edge—during this process. Hence, the graph owner divides the recovered counts by three to obtain the correct numbers of cycle and flow triangles.

9 Security Analysis

We use the simulation paradigm [28] to prove the security guarantees of TriGuard. Our objective is to ensure that an adversary $\mathcal{A}$ with the ability to *statically* corrupt one party cannot obtain any information beyond the claimed leakage defined in Section 4.2. Proving security within the simulation paradigm involves defining two worlds: the *real world*, where the actual protocol is executed by honest parties, and the *ideal world*, where an ideal functionality $\mathcal{F}$ takes inputs from the parties and directly outputs the result to the designated recipient. The adversary $\mathcal{A}$ controls the behavior of corrupted parties and observes their views during protocol execution. To ensure that $\mathcal{A}$ cannot distinguish between the real and ideal worlds, a simulator generates simulated messages to $\mathcal{A}$ that are computationally indistinguishable from those received from honest parties in the real execution. Security is established if, for every efficient adversary $\mathcal{A}$ in the real world, there exists a simulator in the ideal world such that $\mathcal{A}$ cannot distinguish between the two executions. In this setting, the ideal functionality $\mathcal{F}$ accurately captures the leakage function and the claimed leakage profile of our protocols. If $\mathcal{A}$ cannot distinguish between the two worlds, we conclude that TriGuard achieves the claimed security.

Ideal functionality. The ideal functionality $\mathcal{F}$ for triangle counting on streaming graphs is defined as follows.

- Query: The graph owner submits a query $Q = \{W\}$ to $\mathcal{F}$, where W denotes the time window size.
- Append: The graph owner continuously submits newly arrived edges (v, v', t) to $\mathcal{F}$. $\mathcal{F}$ appends these edges to the maintained streaming graph G.
- Count: Upon receiving a query request, $\mathcal{F}$ continuously counts cycle and flow triangles on each snapshot of G.
- Output: $\mathcal{F}$ returns the triangle counts for each snapshot to the graph owner.

Note that in TriGuard, the graph owner only supplies input data and alone obtains the resulting triangle counts. Therefore, our analysis primarily focuses on the scenario where the adversary controls one of the servers. TriGuard allows $\mathcal{F}$ to leak to the servers the following leakage function: $\mathcal{L}(\mathcal{F}(Q)) = \{\{\text{t}\}, |G|, W\}$, where $\{\text{t}\}$ denotes the timestamp of each edge, $|G|$ denotes the total number of edges in the current streaming graph, and W denotes the window size.

Definition 4. *Let $\prod$ denote the protocol for triangle counting, where the graph owner provides the query Q as input. Let $\mathcal{A}$ be an adversary who statically corrupts one server and let $\text{View}^{\prod(Q)}_{\text{Real}}$ denote the view of the corrupted server during the protocol execution. In the ideal world, a simulator generates a simulated view $\text{View}^{\mathcal{L}(\mathcal{F}(Q))}_{\text{Ideal}}$ using only the leakage $\mathcal{L}(\mathcal{F}(Q))$. We say that $\prod$ is secure if there exists a PPT simulator such that $\text{View}^{\mathcal{L}(\mathcal{F}(Q))}_{\text{Ideal}}$ is indistinguishable from $\text{View}^{\prod(Q)}_{\text{Real}}$.*

Proposition 1. *Based on Definition 4, TriGuard securely realizes $\mathcal{F}$ with the leakage $\mathcal{L}(\mathcal{F}(Q))$.*

Proof. Since the roles of the servers in TriGuard are symmetric, it suffices to prove the existence of a simulator for S_1. During the Query and Append phases, S_1 receives only the secret shares of private values in the streaming graph, along with the claimed leakage $(\{\text{t}\}, |G|, W)$. Thus, we can trivially construct its simulator by invoking the RSS simulator [1] with $(\{\text{t}\}, |G|, W)$ as input. In addition, since the authentication process on the servers' side is implemented through the standard secret-shared multiplication procedure, its simulator can also be trivially constructed by invoking the RSS simulator [1]. Moreover, since S_1 learns no information during the Output phase, a simulator for Output trivially exists.

The Count phase comprises four key components: (1) preprocessing the streaming graph (secPrep), (2) secure neighbor aggregation (secAgg), (3) copying neighbor vectors to incident edges (secCopy), and (4) computing triangle counts via neighbor vectors (secCount). Since these components are executed sequentially and their inputs and outputs remain secret-shared, we analyze the existence of their simulators separately, following [7].

Simulator for secPrep**:** At the beginning of secPrep, S_1 holds the secret shares of the edge sequence $\{(\langle v_i \rangle_{\{1,2\}}, \langle v_j \rangle_{\{1,2\}})\}$. S_1 locally converts this edge sequence into a DAG-list. Since the conversion is performed entirely locally, the simulator trivially exists. Therefore, the simulator for oblivConv exists.

Simulator for secAgg**.** We first analyze the simulator for out-neighbor aggregation, i.e., Algorithm 1. At the beginning of Algorithm 1, S_1 holds the secret shares of the DAG-list. During execution, S_1 only receives information from other servers during the message preparation phase (line 1) and the secure sorting phase (lines 2 and 4). Since the message preparation phase involves only basic secret-shared multiplications, its simulator can be trivially constructed by invoking the RSS simulator [1]. Similarly, the simulator for the secure sorting phase can be directly constructed by invoking the simulator of the secure sorting protocol [4], using the number of tuples (i.e., E) as input. Note that the value of E is not an additional leakage, as it can be derived from the time-window size and the public timestamp of each edge. The remaining steps consist solely of local secret-shared additions and subtractions, which require no interaction among servers; thus, their simulators trivially exist. Therefore, a simulator for Algorithm 1 exists. For Algorithm 2, the only steps in which S_1 receives information from other servers are likewise the message preparation phase (line 1) and the secure sorting phase (lines 2 and 4). Consequently, its simulator can also be trivially constructed by invoking the RSS simulator [1] and the secure sorting simulator [4]. Hence, the simulator for secAgg exists.

Simulator for secCopy**.** At the beginning of Algorithm 3, S_1 holds the secret shares of the DAG-list in vertex order. During execution, S_1 receives information from other servers only during the neighbor-copy phase (lines 4–5, 7–8) and the secure sorting phase (lines 3 and 6). Since the neighbor-copy phase involves only basic secret-shared multiplications, its simulator can be trivially constructed by invoking the RSS simulator [1]. Likewise, the simulator for the secure sorting phase can be directly constructed by invoking the simulator of the secure sorting protocol [4], using the number of tuples (i.e., E) as input. The remaining steps consist solely of local secret-shared additions and subtractions, which require no interaction among servers; thus, their simulators trivially exist. Therefore, the simulator for secCopy exists.

Simulator for secCount**.** Since Algorithm 4 consists solely of basic addition, subtraction, and multiplication operations, its simulator can be trivially constructed by invoking the RSS simulator [1]. □

10 Evaluation

Note that TriGuard does not introduce any utility loss into the triangle counting process. Therefore, our experiments focus on evaluating the time and communication costs of TriGuard, rather than assessing its utility on downstream tasks. In our evaluation, we aim to address the following research questions:

- *How does TriGuard perform compared to a baseline constructed using a generic MPC framework?* (Section 10.2)
- *How does the cost of TriGuard increase compared to the DP-based method?* (Section 10.2)
- *What is the performance breakdown of the individual components in TriGuard?* (Section 10.3)

10.1 Experimental Setup

We implement a prototype of TriGuard. All server-side experiments are conducted on a workstation equipped with an Intel Xeon Gold 6238R CPU, 128 GB of RAM, and 1 TB of external SSD storage, running Ubuntu 24.04 LTS. Each server in TriGuard is simulated as a separate single-threaded process. Following previous work [24], server-side communication is emulated locally via the loopback interface, with a bandwidth of 1 Gbps and a latency of 0.5 ms simulated using the Linux `tc` command. In addition, a MacBook Pro with 48 GB of RAM is used as the graph owner, responsible for uploading graph streams and query requests to the workstation. All private values are represented in the ring $\mathbb{Z}_{2^{32}}$.

Datasets. We evaluate TriGuard on four real-world streaming graph datasets of varying scales and structural characteristics, which are commonly used to benchmark graph analytics under dynamic settings. (1) *MOOC User Action*[2] is a small but dense bipartite interaction network. It consists of 7,143 vertices and 411,749 timestamped edges, and exhibits frequent interactions within short time windows. (2) *Reddit Hyperlink Network*[3] is a medium-scale and relatively sparse network. The dataset contains 55,863 vertices and 858,490 temporal edges, reflecting dynamic and bursty interaction patterns. (3) *YouTube Group Memberships Network*[4] is a medium-sized sparse bipartite graph. It comprises 124,325 vertices and 293,360 temporal edges, and represents typical sparsity observed in large-scale social platforms. (4) *com-DBLP*[5] is a large-scale and sparse collaboration network. It includes 317,080 vertices and 1,049,866 edges, and poses significant scalability challenges due to its size and long temporal span.

Baselines. As discussed in Section 1, no prior work directly addresses the specific problem targeted by our study. To facilitate a fair evaluation of our approach, we construct a baseline using the widely adopted generic MPC framework MP-SPDZ [23]. This setup allows us to objectively evaluate the effectiveness of our customized design in TriGuard. In the baseline, we adopt the same system model (i.e., a three-server non-colluding setting) as TriGuard and use the DAG-list representation for the streaming graph.

At a high level, given a secret-shared snapshot graph, the baseline first uses generic secure operations in MP-SPDZ to obliviously enumerate candidate edge tuples that may form triangles. For each candidate tuple, the servers then securely check whether the three edges indeed form a valid directed triangle, and further determine whether it is a cycle triangle or a flow triangle according to the directions of the involved edges. These checks are implemented using the generic arithmetic and comparison primitives in MP-SPDZ, without relying on any graph-specific optimization. Specifically, equal$(\cdot)$ in MP-SPDZ is used to securely test whether two

[2] https://snap.stanford.edu/data/act-mooc.html
[3] https://snap.stanford.edu/data/soc-RedditHyperlinks.html
[4] http://konect.cc/networks/youtube-groupmemberships/
[5] https://snap.stanford.edu/data/com-DBLP.html

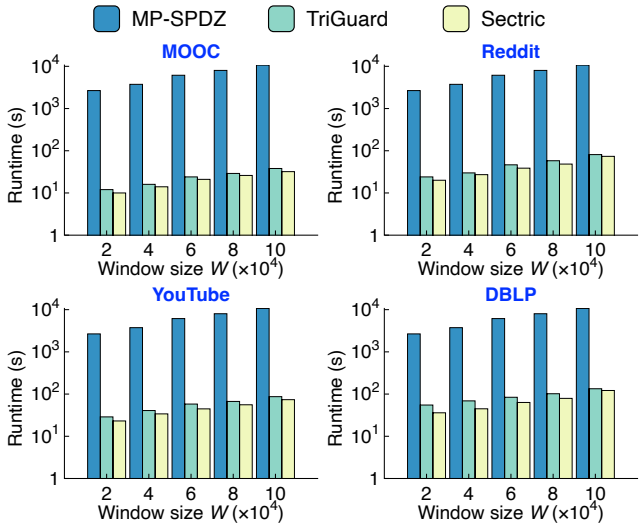


Figure 8: Runtime comparison between TriGuard and the baselines.

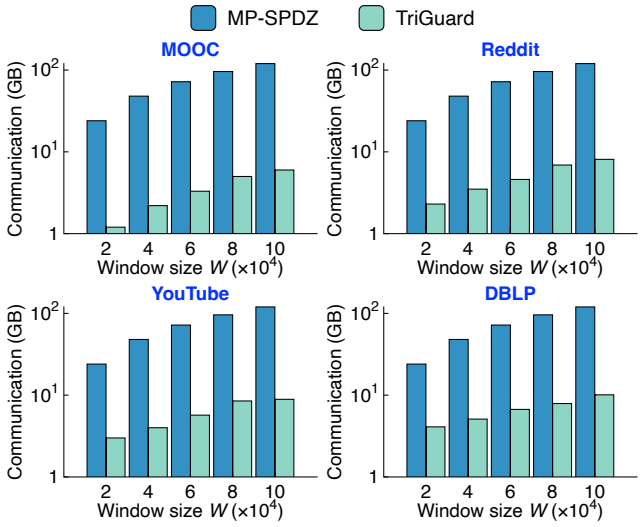


Figure 9: Server-side communication cost comparison between TriGuard and the baseline.

secret-shared vertex IDs are equal, which is necessary for determining whether different edges share common endpoints. add(·) and mul(·) are used to combine intermediate secret-shared indicators and obtain the final secret-shared triangle counts. secure_shuffle(·) and get_random(·) are used to hide access patterns before value recovery, while reveal_to(·) is used only during the recovery of shuffled values.

In addition, we compare TriGuard with Sectric [45], a state-of-the-art method for private triangle counting, using its official implementation[6]. Sectric combines MPC with DP, thereby introducing utility loss into the triangle counting results. Our goal is to demonstrate that, compared with Sectric, TriGuard incurs only modest overhead while achieving *lossless* triangle counting. Since Sectric does not natively support graph streams or distinguish between cycle and flow triangles, we evaluate it using a best-effort adaptation: we apply Sectric independently to each snapshot and ignore edge directions during triangle counting.

10.2 Performance Evaluation and Comparison

Runtime. Fig. 8 compares the runtime of processing each snapshot graph in TriGuard with the two baselines across varying time window sizes $W \in \{2\times10^4, 4\times10^4, 6\times10^4, 8\times10^4, 10\times10^4\}$ on different datasets. The results show that the runtime of the MP-SPDZ baseline is the highest. In contrast, TriGuard consistently outperforms this baseline by a significant margin, achieving speedups of approximately 285× on the MOOC dataset, 131× on the Reddit dataset, 122× on the YouTube dataset, and 80× on the DBLP dataset. This performance gap arises because MP-SPDZ performs exhaustive counting of both cycle and flow triangles, resulting in a time complexity of $O(E^3)$, where E denotes the number of edges in each snapshot graph. In contrast, TriGuard is built upon an efficient oblivious message passing mechanism, which reduces the computational complexity to $O(E \times V)$, where V is the number of vertices in the system—typically much smaller than E. These results highlight the superior scalability of TriGuard and demonstrate the effectiveness of its custom design, which leverages oblivious message passing to achieve substantial performance improvements.

Compared to the DP-based method Sectric, TriGuard introduces only a modest additional computational cost—within roughly 20%. Specifically, compared to Sectric, TriGuard incurs an additional cost of 18.1% on the MOOC dataset, 15.6% on the Reddit dataset, 18.8% on the YouTube dataset, and 20.5% on the DBLP dataset. These results further demonstrate that, even while achieving *lossless* triangle counting with security preservation, TriGuard incurs only a modest runtime overhead relative to Sectric.

Server-side communication. Fig. 9 compares the server-side communication cost of TriGuard with the MP-SPDZ baseline. Note that we do not include Sectric [45] in this comparison, as its communication occurs primarily between the graph owner and the server, making it not directly comparable to our server-side communication model. Consistent with the runtime results, TriGuard achieves substantially lower communication overhead than the MP-SPDZ baseline, with reductions of approximately 95.4% on the MOOC dataset, 93.1% on the Reddit dataset, 92.6% on the YouTube dataset, and 92.5% on the DBLP dataset. This significant reduction stems from our oblivious message passing design, which avoids the heavy interaction and intermediate data exchange required by general-purpose MPC protocols.

10.3 Performance Breakdown of TriGuard

In this section, we analyze the performance contributions of individual components in TriGuard. TriGuard consists of four key components: (1) preprocessing the streaming graph (secPrep), (2) secure neighbor aggregation (secAgg, Section 7.2), (3) copying neighbor vectors to incident edges (secCopy, Section 8), and (4) computing triangle counts via neighbor vectors (secCount, Section 8).

[6]https://github.com/zhentaixie/Sectric

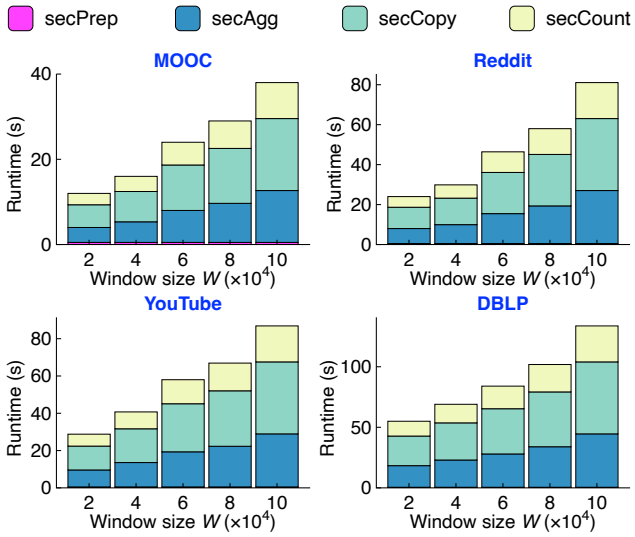


Figure 10: Component-wise runtime breakdown of TriGuard.

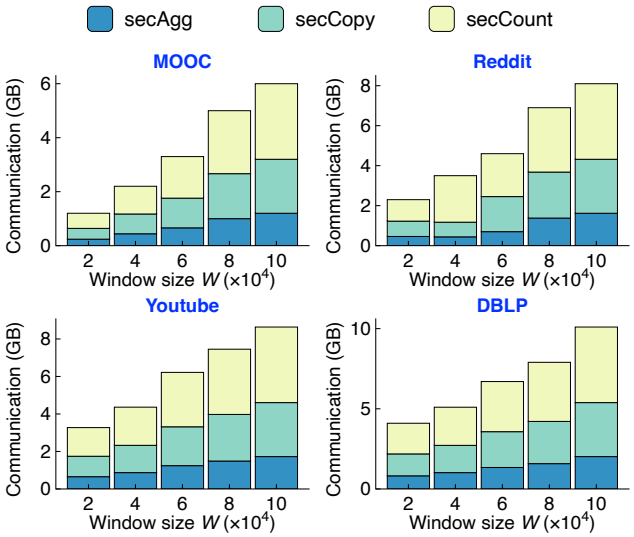


Figure 11: Component-wise communication cost breakdown of TriGuard.

Runtime. We provide a detailed breakdown of the runtime for each component of TriGuard in Fig. 10. The results show that the proportion of time spent on each component remains relatively stable across different settings. This is because the operations in TriGuard are predominantly oblivious, meaning that the underlying private values have minimal impact on runtime performance. Instead, the overall time cost is primarily affected by the dataset characteristics and the time window size. Furthermore, the results indicate that the runtime is dominated by the *secure copying of neighbor vectors to incident edges* (i.e., secCopy). This is because secCopy must propagate the secret-shared in- and out-neighbor vectors of each vertex to its incident edge tuples, so that each edge can locally access the neighbor information of both its source and destination vertices for subsequent triangle counting. To achieve this in an oblivious manner, the servers need to perform secure sorting and multiple element-wise vector multiplications over the one-hot neighbor vectors, which account for the majority of the computational and communication overhead. In contrast, the runtime of secPrep remains negligible, as it mainly consists of local preprocessing operations, such as transforming the edge-list representation into the DAG-list representation and initializing auxiliary fields. These results suggest that future optimizations should focus on reducing the cost of secCopy, for example by fusing neighbor-vector copying with the subsequent counting phase or designing more compact encodings for neighbor vectors to reduce the number of secure vector multiplications.

Server-side communication. We present a detailed breakdown of the server-side communication cost for each component of TriGuard in Fig. 11. Consistent with the runtime analysis, we observe that the proportion of communication overhead attributed to each component remains relatively stable across different settings. This stability arises because the operations in TriGuard are predominantly oblivious, meaning that the underlying private values have minimal influence on communication performance. Furthermore, the results show that the overall communication cost is dominated by the *secure computation of triangle counts via neighbor vectors* (i.e., secCount). In contrast, the preprocessing of the streaming graph (i.e., secPrep) consists solely of local operations and does not involve any server-to-server communication. Therefore, its communication cost is omitted from Fig. 11.

11 Conclusion

We present TriGuard, the first system framework for *oblivious and lossless* triangle counting over streaming graphs. By combining efficient DAG-based graph modeling, lightweight replicated secret sharing, and a tailored oblivious message passing mechanism, TriGuard enables secure, direction-aware triangle counting without revealing edge endpoints, graph topology, or intermediate results. Extensive experiments on real-world streaming graph datasets demonstrate that TriGuard can process each snapshot graph within tens of seconds, even with a time window size of 100,000. Compared to a baseline built on a generic MPC framework, TriGuard achieves up to 285× speedup in runtime and a 95% reduction in communication cost. Furthermore, in contrast to the closest state-of-the-art method, which sacrifices accuracy to enhance privacy, TriGuard achieves lossless accuracy while introducing only approximately 20% additional runtime overhead. We believe that TriGuard takes an important step toward practical and trustworthy secure analytics on dynamic graphs, opening up new possibilities for privacy-preserving graph computation in real-world applications.

References

[1] Toshinori Araki, Jun Furukawa, Yehuda Lindell, Ariel Nof, and Kazuma Ohara. 2016. High-Throughput Semi-Honest Secure Three-Party Computation with an Honest Majority. In *Proc. of ACM CCS*.

[2] Toshinori Araki, Jun Furukawa, Kazuma Ohara, Benny Pinkas, Hanan Rosemarin, and Hikaru Tsuchida. 2021. Secure Graph Analysis at Scale. In *Proc. of ACM CCS*.

[3] ArangoDB. 2024. ArangoGraph-Where Cloud Meets Performance. https://dashboard.arangodb.cloud/home.

[4] Gilad Asharov, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Ariel Nof, Benny Pinkas, Katsumi Takahashi, and Junichi Tomida. 2022. Efficient secure three-party sorting with applications to data analysis and heavy hitters. In *Proc. of ACM CCS*.

[5] Luca Becchetti, Paolo Boldi, Carlos Castillo, and Aristides Gionis. 2008. Efficient semi-streaming algorithms for local triangle counting in massive graphs. In *Proc. of ACM SIGKDD*. 16–24.

[6] James Bell, Adria Gascon, Badih Ghazi, Ravi Kumar, Pasin Manurangsi, Mariana Raykova, and Phillipp Schoppmann. 2022. Distributed, Private, Sparse Histograms in the Two-Server Model. In *Proc. of ACM CCS*.

[7] Max Curran, Xiao Liang, Himanshu Gupta, Omkant Pandey, and Samir R Das. 2019. Procsa: Protecting privacy in crowdsourced spectrum allocation. In *Proc. of ESORICS*.

[8] Emma Dauterman, Mayank Rathee, Raluca Ada Popa, and Ion Stoica. 2022. Waldo: A Private Time-Series Database from Function Secret Sharing. In *Proc. of IEEE S&P*.

[9] Cynthia Dwork, Aaron Roth, et al. 2014. The algorithmic foundations of differential privacy. *Foundations and trends® in theoretical computer science* 9, 3–4 (2014), 211–407.

[10] Francesca Falzon, Esha Ghosh, Kenneth G Paterson, and Roberto Tamassia. 2024. PathGES: An Efficient and Secure Graph Encryption Scheme for Shortest Path Queries. In *Proc. of ACM CCS*.

[11] Fireblocks. 2025. Powering the Digital Asset Economy. online at https://www.fireblocks.com. [Online; Accessed 9-Jul-2025].

[12] Esha Ghosh, Seny Kamara, and Roberto Tamassia. 2021. Efficient Graph Encryption Scheme for Shortest Path Queries. In *Proc. of ACM AsiaCCS*.

[13] Google Cloud. 2023. Streaming graph data with Confluent Cloud and Neo4j on Google Cloud. https://cloud.google.com/blog/products/data-analytics/keep-graph-data-updated-on-google-cloud-using-confluent--neo4j.

[14] Xiangyang Gou and Lei Zou. 2021. Sliding window-based approximate triangle counting over streaming graphs with duplicate edges. In *Proc. of ACM SIGMOD*.

[15] Xiangyang Gou and Lei Zou. 2023. Sliding window-based approximate triangle counting with bounded memory usage. *The VLDB Journal* 32, 5 (2023), 1087–1110.

[16] Yizhang He, Kai Wang, Wenjie Zhang, Xuemin Lin, Ying Zhang, and Wei Ni. 2025. Robust Privacy-Preserving Triangle Counting under Edge Local Differential Privacy. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26.

[17] Kai Huang, Haibo Hu, Shuigeng Zhou, Jihong Guan, Qingqing Ye, and Xiaofang Zhou. 2022. Privacy and efficiency guaranteed social subgraph matching. *The VLDB Journal* 31, 3 (2022), 581–602.

[18] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2021. Locally differentially private analysis of graph statistics. In *Proc. of USENIX Security*. 983–1000.

[19] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2022. Communication-Efficient triangle counting under local differential privacy. In *Proc. of USENIX Security*.

[20] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2022. Differentially private triangle and 4-cycle counting in the shuffle model. In *Proc. of ACM CCS*.

[21] Peipei Jiang, Qian Wang, Muqi Huang, Cong Wang, Qi Li, Chao Shen, and Kui Ren. 2021. Building In-the-Cloud Network Functions: Security and Privacy Challenges. *Proc. IEEE* 109, 12 (2021), 1888–1919.

[22] Peipei Jiang, Qian Wang, Muqi Huang, Cong Wang, Qi Li, Chao Shen, and Kui Ren. 2021. Building in-the-cloud network functions: Security and privacy challenges. *Proc. IEEE* 109, 12 (2021), 1888–1919.

[23] Marcel Keller. 2020. MP-SPDZ: A versatile framework for multi-party computation. In *Proc. of ACM CCS*.

[24] Nishat Koti, Varsha Bhat Kukkala, Arpita Patra, and Bhavish Raj Gopal. 2024. Graphiti: Secure Graph Computation Made More Scalable. In *Proc. of ACM CCS*.

[25] Youhuan Li, Lei Zou, M Tamer Özsu, and Dongyan Zhao. 2019. Time constrained continuous subgraph search over streaming graphs. In *Proc. of IEEE ICDE*. 1082–1093.

[26] Xuankun Liao, Qing Liu, Xin Huang, and Jianliang Xu. 2024. Truss-based community search over streaming directed graphs. *Proceedings of the VLDB Endowment* 17, 8 (2024), 1816–1829.

[27] Yongsub Lim, Minsoo Jung, and U Kang. 2018. Memory-efficient and accurate sampling for counting local triangles in graph streams: From simple to multigraphs. *ACM Transactions on Knowledge Discovery from Data* 12, 1 (2018), 1–28.

[28] Yehuda Lindell. 2017. How to Simulate It - A Tutorial on the Simulation Proof Technique. In *Tutorials on the Foundations of Cryptography*. 277–346.

[29] Qing Liu, Minjun Zhao, Xin Huang, Jianliang Xu, and Yunjun Gao. 2020. Truss-based community search over large directed graphs. In *Proc. of ACM SIGMOD*.

[30] Shang Liu, Yang Cao, Takao Murakami, Jinfei Liu, and Masatoshi Yoshikawa. 2024. CARGO: Crypto-Assisted Differentially Private Triangle Counting without Trusted Servers. In *Proc. of IEEE ICDE*. 1671–1684.

[31] Meta. 2025. The value of secure multi-party computation. online at https://privacytech.fb.com/multi-party-computation/. [Online; Accessed 9-Jul-2025].

[32] MPCVault. 2025. Multisig crypto wallet for business. online at https://mpcvault.com. [Online; Accessed 9-Jul-2025].

[33] NEO4J CLOUD. 2025. The world's most popular graph data platform, available for any cloud environment. https://neo4j.com/cloud/.

[34] Anil Pacaci, Angela Bonifati, and M Tamer Özsu. 2020. Regular path query evaluation on streaming graphs. In *Proc. of ACM SIGMOD*. 1415–1430.

[35] Marco Sánchez-Aguayo, Luis Urquiza-Aguiar, and José Estrada-Jiménez. 2021. Fraud detection using the fraud triangle theory and data mining techniques: A literature review. *Computers* 10, 10 (2021), 121.

[36] Sijun Tan, Brian Knott, Yuan Tian, and David J Wu. 2021. CryptGPU: Fast Privacy-Preserving Machine Learning on the GPU. In *Proc. of IEEE S&P*.

[37] Chenghong Wang, Johes Bater, Kartik Nayak, and Ashwin Machanavajjhala. 2022. IncShrink: architecting efficient outsourced databases using incremental mpc and differential privacy. In *Proc. of ACM SIGMOD*.

[38] Songlei Wang, Yifeng Zheng, and Xiaohua Jia. 2024. GraphGuard: Private Time-Constrained Pattern Detection Over Streaming Graphs in the Cloud. In *Proc. of USENIX Security*.

[39] Songlei Wang, Yifeng Zheng, Xiaohua Jia, Qian Wang, and Cong Wang. 2023. MAGO: Maliciously secure subgraph counting on decentralized social graphs. *IEEE Transactions on Information Forensics and Security* 18 (2023), 2929–2944.

[40] Songlei Wang, Yifeng Zheng, Yi Zhang, Yuchuan Luo, and Cong Wang. 2026. The full version of TriGuard. https://github.com/songleiW/Cove-code/blob/main/Cove_full_version.pdf. [Online; Accessed 25-Jun-2026].

[41] Siyue Wu, Dingming Wu, Sinhong Cheuk, Tsz Nam Chan, and Kezhong Lu. 2025. GREAT: Generalized Reservoir Sampling based Triangle Counting Estimation over Streaming Graphs. *Proceedings of the VLDB Endowment* 18, 7 (2025), 2031–2043.

[42] Yingying Wu, Jiabei Wang, Dandan Xu, and Yongbin Zhou. 2024. Spidey: Secure Dynamic Encrypted Property Graph Search With Lightweight Access Control. *IEEE Internet of Things Journal* (2024).

[43] Yuyang Xia, Yixiang Fang, and Wensheng Luo. 2025. Efficiently Counting Triangles in Large Temporal Graphs. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–27.

[44] Lyu Xu, Byron Choi, Yun Peng, Jianliang Xu, and Sourav S Bhowmick. 2023. A Framework for Privacy Preserving Localized Graph Pattern Query Processing. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–27.

[45] Minze Xu, Zhentai Xie, Zhibin Wang, Guangzhan Wang, Longbin Lai, Yuan Zhang, Chen Tian, and Sheng Zhong. 2025. Sectric: Towards Accurate, Privacy-preserving and Efficient Triangle Counting. *Proceedings of the VLDB Endowment* 18, 10 (2025), 3382–3395.

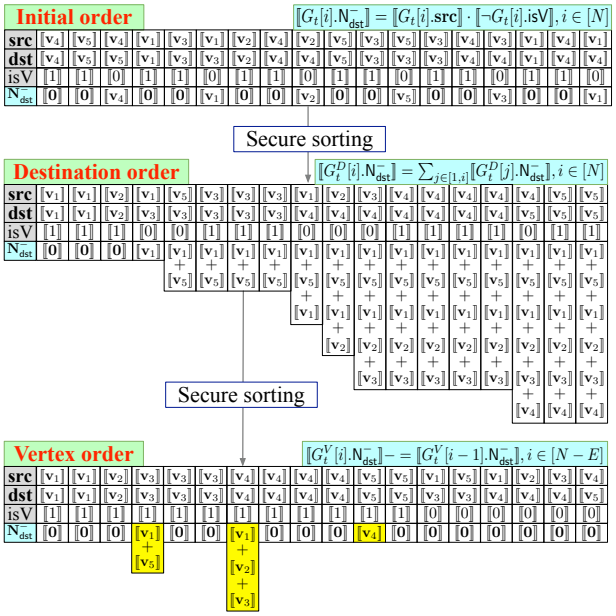


Figure 12: Illustration of secure in-neighbor aggregation on snapshot G_6 in Fig. 2.

A Example of Secure In-neighbor Aggregation

Fig. 12 illustrates the process of secure in-neighbor aggregation on the snapshot G_6 from Fig. 2. The input is a DAG-list in its initial order. Note that the input may be the output of secNout, i.e., in vertex order; however, the ordering can be arbitrary and does not affect the correctness of the result. Each edge tuple's $\mathbf{N}^{+}_{\text{dst}}$ is initialized to its corresponding **src** value and treated as the message to be propagated to the tuple corresponding to its destination vertex, i.e., the in-neighbors. The DAG-list is then sorted into the destination order, enabling the servers to aggregate the $\mathbf{N}^{-}_{\text{dst}}$ from preceding tuples into each tuple's own $\mathbf{N}^{-}_{\text{dst}}$. Subsequently, the DAG-list is sorted from the destination order into the vertex order, allowing to eliminate redundant entries and obtain the correct in-neighbors for each vertex. The resulting output shows that each vertex tuple's $\mathbf{N}^{-}_{\text{dst}}$ stores its in-neighbors, i.e., $\mathbf{N}^{-}_{v_3} = \mathbf{v}_1 + \mathbf{v}_5$, $\mathbf{N}^{-}_{v_4} = \mathbf{v}_1 + \mathbf{v}_2 + \mathbf{v}_3$, and $\mathbf{N}^{-}_{v_5} = \mathbf{v}_4$.

B Example of Secure Neighbor Vector Copying

Fig. 13 illustrates the process of copying neighbor vectors to incident edges on snapshot G_6 shown in Fig. 2. The input is a DAG-list ordered by vertices, where for each vertex tuple, both $\mathbf{N}^{+}_{\text{src}}$ and $\mathbf{N}^{+}_{\text{dst}}$ store the vertex's out-neighbor vector, and both $\mathbf{N}^{-}_{\text{src}}$ and $\mathbf{N}^{-}_{\text{dst}}$ store its in-neighbor vector. The servers first sort the DAG-list into source order, enabling them to obliviously propagate each vertex's out- and in-neighbor vectors to the corresponding outgoing edges' $\mathbf{N}^{+}_{\text{src}}$ and $\mathbf{N}^{-}_{\text{src}}$ fields. This oblivious copy operation is implemented by aggregating each tuple's $\mathbf{N}^{+}_{\text{src}}$ and $\mathbf{N}^{-}_{\text{src}}$ with those of its succeeding tuple. Next, the servers sort the DAG-list into destination order, which allows them to propagate each vertex's out- and in-neighbor vectors to the corresponding incoming edges' $\mathbf{N}^{+}_{\text{dst}}$ and $\mathbf{N}^{-}_{\text{dst}}$ fields. As a result, each edge tuple obtains the neighbor vectors of both its source and destination vertices. For example, for edge $e_{1,3}$, $\mathbf{N}^{+}_{v_1} = \mathbf{v}_3 + \mathbf{v}_4$; for edge $e_{5,3}$, $\mathbf{N}^{+}_{v_5} = \mathbf{v}_3$, $\mathbf{N}^{+}_{v_3} = \mathbf{v}_4$, $\mathbf{N}{v_5}^{-} = \mathbf{v}_4$, and $\mathbf{N}^{-}_{v_3} = \mathbf{v}_1 + \mathbf{v}_5$.

Figure 13: Illustration of securely copying neighbor vectors to incident edges on snapshot G_6 from Fig. 2.